Fundamentos de Programación

Grado en Ingeniería del Software

15/10/26

Antecedentes

- Estructura básica de un programa
- Descomposición de problemas en funciones
- Lectura de ficheros
- Esquemas de filtrado/transformación
- Paso de parámetros
- Type hinting

Índice

- Filtrado y transformación por comprensión
- Suma
- Contador
- Existe
- Para todo
- Máximo y mínimo por defecto
- Ordenación por defecto

FILTRADO Y TRANSFORMACIÓN POR COMPRENSIÓN

```python
def consonants_for(sentence):
    """Convierte todos los caracteres de la frase a mayúsculas usando un bucle for."""
    result = []
    for x in sentence:
        result.append(x.upper())
    return result


def consonants_lc(sentence):
    """Convierte todos los caracteres de la frase a mayúsculas usando list comprehension."""
    return [x.upper() for x in sentence]

if __name__ == "__main__":
    sentence = 'we are studying list comprehensions'
    print("With For Loop           : " + ''.join(consonants_for(sentence)))
    print("With List Comprehension : " + ''.join(consonants_lc(sentence)))
```

```
With For Loop           : WE ARE STUDYING LIST COMPREHENSIONS
With List Comprehension : WE ARE STUDYING LIST COMPREHENSIONS
```

```python
# consonants.py
# Comparación entre bucle for y list comprehension para obtener consonantes

VOWELS = 'aeiou'

def consonants_for(sentence):
    """Devuelve una lista con las consonantes de la frase usando un bucle for."""
    result = []
    for x in sentence:
        if x.lower() not in VOWELS and x.isalpha():
            result.append(x)
    return result

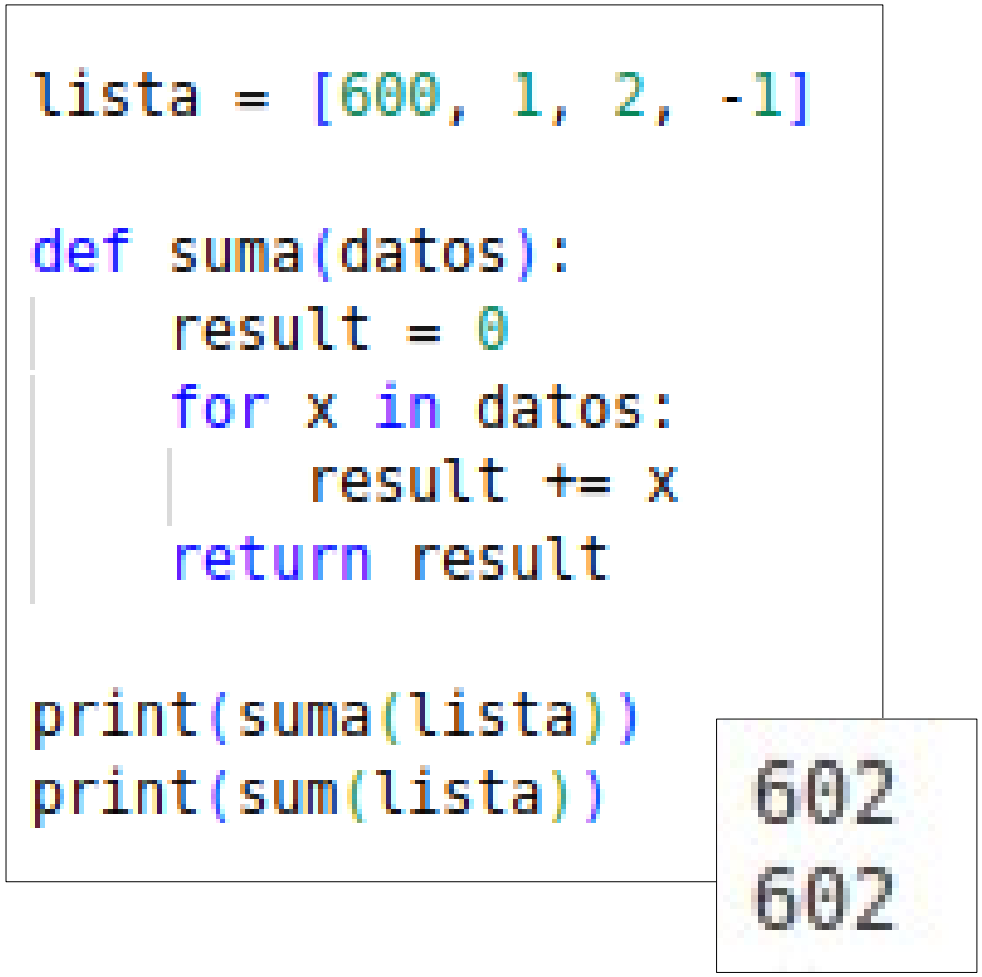

def consonants_lc(sentence):
    """Devuelve una lista con las consonantes de la frase usando list comprehension."""
    return [x for x in sentence if x.lower() not in VOWELS and x.isalpha()]


if __name__ == "__main__":
    sentence = 'we are studying list comprehensions'

    print("With For Loop           : " + ''.join(consonants_for(sentence)))
    print("With List Comprehension : " + ''.join(consonants_lc(sentence)))
```

```
With For Loop           : wrstdynglstcmprhnsns
With List Comprehension : wrstdynglstcmprhnsns
```

SUMA


lista = [600, 1, 2, -1]
def suma(datos):
result = 0
for x in datos:
result += x
return result
print(suma(lista))
print(sum(lista))
602
602

```python
from collections import namedtuple


Persona = namedtuple('Persona', 'nombre, apellidos, edad')

def suma_edades(personas: list[Persona]) -> int:
    result = 0
    for p in personas:
        result += p.edad
    return result

def mapea_edades(personas: list[Persona]) -> list[int]:
    result = []
    for p in personas:
        result.append(p.edad)
    return result

def reduce_suma(edades: list[int]) -> int:
    return sum(edades)

def suma_edades2(personas: list[Persona]) -> int:
    edades = mapea_edades(personas)    # map: cada persona -> su edad
    return reduce_suma(edades)                              # reduce: se suman todas

personas = [
    Persona('Ana', 'García López', 34),
    Persona('Luis', 'Pérez Ruiz', 27),
    Persona('Marta', 'Sánchez Gil', 45),
]
print(suma_edades(personas))
print(suma_edades2(personas))
```

```
106
106
```

```python
def suma_edades(personas: list[Persona]) -> int:
    result: int = 0
    for p in personas:
        result += p.edad
    return result
```

Transformación: $p \in$ Persona, $y = f(p) = p.edad$

Reducción: $e = (e_0, e_1, \ldots, e_{n-1}) \rightarrow y = \sum^{n-1} e_i$

```python
def mapea_edades(personas: list[Persona]) -> list[int]:
    result = []
    for p in personas:
        result.append(p.edad)
    return result

def reduce_suma(edades: list[int]) -> int:
    return sum(edades)

def suma_edades2(personas: list[Persona]) -> int:
    edades = mapea_edades(personas)    # map: cada perso
    return reduce_suma(edades)
```

CONTADOR

```python
def contador_naive(datos):
    count = 0
    for _ in datos:
        count += 1
    return count

sentence = 'we are studying list comprehensions'
print(contador_naive(sentence))
print(len(sentence))
```

```
35
35
```

```python
VOWELS = 'aeiou'
sentence = 'we are studying list comprehensions'

#Ejemplo 1
def contador(sentence: str) -> int:
    result = 0
    for x in sentence:
        if x in VOWELS:
            result += 1
    return result

#Ejemplo 2
def mapea(secuencia: str) -> list[int]:
    result = []
    for x in secuencia:
        result.append(1 if x in VOWELS else 0)
    return result

def contador_2(sentence: str) -> int:
    unos_y_ceros = mapea(sentence)    # map: cada carácter -> 1 o 0
    return reduce_suma(unos_y_ceros)        # reduce: se suman todos

print(contador(sentence))
print(contador_2(sentence))
```

```python
#Ejemplo 1
def contador(sentence):
    result = 0
    for x in sentence:
        if x in VOWELS:
            result += 1
    return result
```

```python
#Ejemplo 2
def mapea(secuencia: str) -> list[int]:
    result = []
    for x in secuencia:
        result.append(1 if x in VOWELS else 0)
    return result

def contador_2(sentence: str) -> int:
    unos_y_ceros = mapea(sentence)    # map: cad
    return reduce_suma(unos_y_ceros)          # re
```

EXISTE

```python
def existe_naive(datos: list[bool])->bool:
    result = False
    for x in datos:
        result = result or x
        if result:
            break
    return result

print(existe_naive([False, True]))
print(any([False, True]))
```

```python
#Ejemplo 1
def existe_vowel(sentence: str) -> bool:
    result = False
    for x in sentence:
        result = result or x in VOWELS
        if result:
            break
    return result

#Ejemplo 2
def mapea_vocal(secuencia: str) -> list[bool]:
    result = []
    for x in secuencia:
        result.append(x in VOWELS)
    return result

def reduce_existe(booleanos: list[bool]) -> bool:
    return any(booleanos)

def existe_vowel2(sentence: str) -> bool:
    booleanos = mapea_vocal(sentence)    # map: cada carácter -> True o False
    return reduce_existe(booleanos)       # reduce: basta con que uno sea True

print(existe_naive([False, True]))
print(any([False, True]))
print(existe_vowel(sentence))
print(existe_vowel2(sentence))
```

```python
#Ejemplo 1
def existe_vowel(sentence: str) -> bool:
    result = False
    for x in sentence:
        result = result or x in VOWELS
        if result:
            break
    return result
```

```python
#Ejemplo 2
def mapea_vocal(secuencia: str) -> list[bool]:
    result = []
    for x in secuencia:
        result.append(x in VOWELS)
    return result

def reduce_existe(booleanos: list[bool]) -> bool:
    return any(booleanos)

def existe_vowel2(sentence: str) -> bool:
    booleanos = mapea_vocal(sentence)    # map: cad
    return reduce_existe(booleanos)       # reduce:
```

PARA TODO

```python
def para_todo_naive(datos: list[bool])->bool:
    result = True
    for x in datos:
        result = result and x
        if not result:
            break
    return result

print(para_todo_naive([False, True]))
print(all([False, True]))
```

```python
#Ejemplo 1
def para_todo_vowel(sentence: str) -> bool:
    result = True
    for x in sentence:
        result = result and x in VOWELS
        if not result:
            break
    return result

#Ejemplo 2
def reduce_para_todo(booleanos: list[bool]) -> bool:
    return all(booleanos)

def para_todo_vowel2(sentence: str) -> bool:
    booleanos = mapea_vocal(sentence)    # map: cada carácter -> True o False
    return reduce_para_todo(booleanos)       # reduce: basta con que uno sea True

print(para_todo_naive([False, True]))
print(all([False, True]))
```

```python
#Ejemplo 1
def para_todo_vowel(sentence: str) -> bool:
    result = True
    for x in sentence:
        result = result and x in VOWELS
        if not result:
            break
    return result
```

```python
#Ejemplo 2
def reduce_para_todo(booleanos: list[bool]) -> bool:
    return all(booleanos)

def para_todo_vowel2(sentence: str) -> bool:
    booleanos = mapea_vocal(sentence)    # map: cada carácter
    return reduce_para_todo(booleanos)       # reduce: basta c
```

MÁXIMO Y MÍNIMO POR DEFECTO

```python
lista = [600, 1, 2, -1]

def max_(datos):
    result = None
    for x in datos:
        if result == None or result<x:
            result = x
    return result

def min_(datos):
    result = None
    for x in datos:
        if result == None or result>x:
            result = x
    return result

print(max(lista), max_(lista), min(lista), min_(lista))
```

```python
def max_persona(personas: list[Persona]) -> Persona | None:
    result = None
    for p in personas:
        if result == None or result.edad<p.edad:
            result = p
    return result

def min_persona(personas: list[Persona]) -> Persona | None:
    result = None
    for p in personas:
        if result == None or result.edad>p.edad:
            result = p
    return result
```

```python
def mapea_edad_persona(personas: list[Persona])-> list[tuple[int, Persona]]:
    result = []
    for p in personas:
        result.append((p.edad, p))
    return result

def max_persona_2(personas: list[Persona]) -> Persona | None:
    tuplas = mapea_edad_persona(personas)

    result = max(tuplas)
    if result is not None:
        result = result[1]

    return result

def min_persona_2(personas: list[Persona]) -> Persona | None:
    tuplas = mapea_edad_persona(personas)

    result = min(tuplas)
    if result is not None:
        result = result[1]

    return result
```

ORDENACIÓN POR DEFECTO

Array Sorting Algorithms

Algorithm	Time Complexity			Space Complexity
	Best	Average	Worst	Worst
Quicksort	Ω(n log(n))	θ(n log(n))	O(n^2)	O(log(n))
Mergesort	Ω(n log(n))	θ(n log(n))	O(n log(n))	O(n)
Timsort	Ω(n)	θ(n log(n))	O(n log(n))	O(n)
Heapsort	Ω(n log(n))	θ(n log(n))	O(n log(n))	O(1)
Bubble Sort	Ω(n)	θ(n^2)	O(n^2)	O(1)
Insertion Sort	Ω(n)	θ(n^2)	O(n^2)	O(1)
Selection Sort	Ω(n^2)	θ(n^2)	O(n^2)	O(1)
Tree Sort	Ω(n log(n))	θ(n log(n))	O(n^2)	O(n)
Shell Sort	Ω(n log(n))	θ(n(log(n))^2)	O(n(log(n))^2)	O(1)
Bucket Sort	Ω(n+k)	θ(n+k)	O(n^2)	O(n)
Radix Sort	Ω(nk)	θ(nk)	O(nk)	O(n+k)
Counting Sort	Ω(n+k)	θ(n+k)	O(n+k)	O(k)
Cubesort	Ω(n)	θ(n log(n))	O(n log(n))	O(n)

```
>>> array = [8, 2, 6, 4, 5]
>>> sorted(array)
[2, 4, 5, 6, 8]
```

TAREA

- Notebook 3
- Completar Parte 0 y 1 de Aeropuerto.